Create test/support/factory/sms_message.ex and add the following code:

phone_app/test/support/factory/sms_message.ex

```
defmodule Test.Factory.SmsMessageFactory do
  def params(overrides \\ %{}) do
    %{
      from: "+11112223333",
      to: "+19998887777",
      direction: :outgoing,
      message_sid: Ecto.UUID.generate(),
      account_sid: "account_sid",
      body: "body",
      status: "queued"
    }
    |> Map.merge(overrides)
  end

  def create(overrides \\ %{}) do
    {:ok, message} =
      overrides
      |> params()
      |> PhoneApp.Conversations.create_sms_message()

    message
  end
end
```

ExUnit is configured by the Phoenix generator to include code inside of test/support during test execution. This will make the factory available to all of our test files.

There are factory libraries in Elixir, but my personal preference is to not use them. Instead, create factory modules and explicitly set up dependencies as needed.

Set Up Your Test Module

Create test/phone_app_web/controllers/message_controller_test.exs and add the following skeleton:

phone_app/test/phone_app_web/controllers/message_controller_test.exs

```
defmodule PhoneAppWeb.MessageControllerTest do
  use PhoneAppWeb.ConnCase, async: true

  alias Test.Factory.SmsMessageFactory
end
```

This skeleton forms the basis of our controller test. We use PhoneAppWeb.ConnCase this time. You'll find that module at test/support/conn_case.ex. The ConnCase module is created by Phoenix and sets up a conn dependency as well as the same dependencies as DataCase.